

UNIVERSITÀ DEGLI STUDI DI MILANO
FACOLTÀ DI SCIENZE E TECNOLOGIE

DIPARTIMENTO DI INFORMATICA
“GIOVANNI DEGLI ANTONI”



LAUREA MAGISTRALE IN
SICUREZZA INFORMATICA

TODO: TITOLO DELLA TESI

Autore: Enea Manzi
Matricola: 65935A

Relatore: Prof. Marco Anisetti
Correlatore: Prof. Claudio Agostino Ardagna

A.A. 2025-2026

Abstract

Ringraziamenti

Indice

Abstract	ii
Ringraziamenti	iii
Elenco delle figure	v
Elenco delle tabelle	vi
Elenco dei codici	vii
1 Introduzione	1
1.1 Motivazione e gaps	1
1.2 Obiettivi	1
1.3 Struttura della tesi	1
2 Background e Stato dell'Arte	2
2.1 Esempio di sezione	2
3 Metodologia, Architettura e Principi di Design	3
3.1 Principi Fondamentali: il Perché delle Scelte Architettureali	3
3.1.1 Agnosticismo Applicativo (D1.P1)	4
3.1.2 La Specifica OpenAPI come Unico Oracolo Formale (D3.P2)	5
3.1.3 Config-Driven Development (D3.P1)	6
3.1.4 Riproducibilità Deterministica (D3.P4)	6
3.2 Architettura dell'Assessment Engine	7
3.2.1 Flusso di Dipendenze Monodirezionale (D1.P2)	7
3.2.2 Scheduling Basato su DAG e Orchestrazione Deterministica (D2.P2)	8
3.2.3 Split State e Immutabilità (D1.P3)	9
3.2.4 Gateway-Agnostic Adapter (D1.P6)	10
3.2.5 Streaming Evidence Store (D4.P1)	11
3.2.6 Assenza di Dipendenze di Stato Esterno (D3.P5)	12
3.3 Domini di Sicurezza e Box Gradient Applicato	12
3.3.1 Tassonomia degli Otto Domini di Assurance	13
3.3.2 Il Box Gradient come Mapping dalle Precondizioni al Privilegio (D3.P3)	14

4	Implementazione	17
4.1	La Pipeline di Esecuzione a Sette Fasi e il Core Engine	17
4.1.1	Fail-Safe Error Isolation (D4.P3)	19
4.1.2	Gerarchia delle Eccezioni con Phase Mapping (D4.P4)	19
4.2	Discovery Dinamico e Risoluzione del Grafo	20
4.2.1	Discovery Dinamico via <code>pkgutil</code> (D2.P1)	21
4.2.2	Scheduling Topologico e <code>DAGScheduler</code> (D2.P2)	21
4.2.3	Invarianti di <code>TestResult</code> a Livello di Tipo (D4.P9)	22
4.3	Architettura dei Test Nativi e Contratto <code>BaseTest</code>	23
4.3.1	Gerarchia Duale <code>BaseTest</code> / <code>ExternalToolTest</code> (D1.P4)	23
4.3.2	Auth Abstraction Layer e Idempotenza dei Token (D2.P5)	24
4.3.3	Tracciabilità Normativa e Distinzione Finding/InfoNote (D5.P1, D5.P2)	25
4.3.4	Safe HTTP Probing Policy e Oracle a Tre Esiti (D4.P7)	26
4.4	Gerarchia dei Connettori e Integrazione Tool Esterni	27
4.4.1	Gerarchia a Tre Livelli e Separazione Dati/Valutazione (D1.P5, D2.P3)	27
4.4.2	Lifecycle dei Connector e Dependency Injection (D2.P4)	28
4.4.3	Classificazione A/B, Path Seed e Catalogo dei Payload (D2.P6, D2.P7, D2.P8)	28
4.4.4	Isolamento della Dipendenza AGPL (D7.P2)	29
4.5	Implementazione del Gateway Adapter e Teardown LIFO	30
4.5.1	<code>KongGatewayAdapter</code> : Accesso Read-Only all'Admin API	30
4.5.2	Degradazione Controllata a Tre Livelli (D4.P5)	31
4.5.3	Teardown Best-Effort in Ordine LIFO (D4.P6)	31
4.5.4	Astrazione Trasparente dell'URL di Deployment (D7.P3)	32
4.6	Evidence Store e Sanitizzazione delle Credenziali	32
4.6.1	Ciclo di Vita dello Streaming Evidence Store (D4.P1)	33
4.6.2	Sanitizzazione Centralizzata delle Credenziali (D4.P2)	34
4.6.3	Dual Audit Trail e Report Domain-Centric (D5.P5)	35
5	Validazione sperimentale e risultati	36
5.1	Setup sperimentale	36
5.2	Risultati	36
6	Discussione e Sviluppi Futuri	37
7	Conclusioni	38
7.1	Conclusioni	38
7.2	Lavori futuri	38
	Bibliografia	40

Elenco delle figure

Elenco delle tabelle

Tabella 3.1	Mappa tassonomica delle 39 proprietà architetture di APIGuard Assurance, distribuite sui sette domini di design. Le proprietà discusse principalmente in questo capitolo sono marcate in grassetto; le restanti ricevono trattamento nel Capitolo 4.	4
Tabella 3.2	I otto Domini di Assurance di APIGuard Assurance e il loro perimetro di garanzie. Domain-5 è architetturealmente predisposto ma implementato nella Milestone 2.	13
Tabella 3.3	Mapping tra priorità di test, strategia operativa e prerequisiti tipici in APIGuard Assurance (D3.P3). La colonna Prerequisiti indica le risorse che l'operatore deve fornire prima di avviare i test di quel livello. Il mapping è una raccomandazione strutturale, non un vincolo imposto dal framework: deviazioni architetturealmente giustificate esistono nel codebase.	15
Tabella 4.1	Le sette fasi della pipeline di assessment di APIGuard Assurance. Le fasi 1, 2 e 4 sono bloccanti: un errore interrompe lo startup prima dell'esecuzione dei test. Le fasi 5, 6 e 7 sono non-bloccanti: i fallimenti vengono registrati e l'esecuzione prosegue.	18
Tabella 4.2	Le 11 classi di eccezione custom di APIGuard Assurance, distribuite su 3 file sorgente. Le eccezioni di Phase 1–4 sono fatali e interrompono lo startup; quelle di Phase 5–6 vengono recuperate localmente. Le eccezioni Command-Line Interface (CLI) non transitano dall'engine e sono architetturealmente isolate dal loop di assessment.	20

Elenco dei codici

Capitolo 1

Introduzione

1.1 Motivazione e gaps

1.2 Obiettivi

1.3 Struttura della tesi

Capitolo 2

Background e Stato dell'Arte

2.1 Esempio di sezione

Come discusso in letteratura [1], ...

Capitolo 3

Metodologia, Architettura e Principi di Design

Il passaggio dal problema all'architettura non è mai immediato. Tra il riconoscimento di un gap nella sicurezza delle API e la costruzione di un sistema che lo colma si interpone un insieme di decisioni di design, alcune vincolate da requisiti espliciti, altre derivate da scelte pregresse, altre ancora orientate a garantire che il sistema rimanga valido oltre il contesto specifico della sua prima validazione. Questo capitolo documenta quelle decisioni nell'ordine logico in cui si condizionano a vicenda: prima i principi fondativi, da cui discendono tutte le scelte successive; poi la struttura architetturale del motore di assessment; infine la tassonomia dei domini di sicurezza e il suo legame con i livelli di visibilità del tester.

Il design di APIGuard Assurance è formalizzato in 39 proprietà architetture, distribuite su sette categorie. La Tabella 3.1 ne costituisce la mappa di riferimento per l'intero capitolo e per il successivo. Ogni proprietà riceve trattamento in profondità nella sezione in cui il suo ruolo è primario; il lettore che intende rintracciare la realizzazione implementativa di una specifica proprietà può utilizzare la tabella come indice e seguire il rimando al Capitolo 4, dove le scelte architetture qui descritte trovano traduzione in codice Python.

3.1 Principi Fondamentali: il Perché delle Scelte Architetture

Quattro proprietà architetture precedono tutte le altre non per importanza astratta, ma perché ogni altra scelta del sistema le presuppone. Non descrivono come i componenti funzionano: descrivono perché il sistema è stato progettato in questa forma e non in

Tabella 3.1: Mappa tassonomica delle 39 proprietà architetture di APIGuard Assurance, distribuite sui sette domini di design. Le proprietà discusse principalmente in questo capitolo sono marcate in grassetto; le restanti ricevono trattamento nel Capitolo 4.

Dominio	Proprietà
D1: Architettura & Design	D1.P1 , D1.P2 , D1.P3 , D1.P4, D1.P5, D1.P6
D2: Estensibilità & Manutenibilità	D2.P1, D2.P2 , D2.P3, D2.P4, D2.P5, D2.P6, D2.P7, D2.P8
D3: Config & Riproducibilità	D3.P1 , D3.P2 , D3.P3 , D3.P4 , D3.P5
D4: Robustezza & Sicurezza	D4.P1 , D4.P2, D4.P3, D4.P4, D4.P5, D4.P6, D4.P7, D4.P8, D4.P9
D5: Qualità & Osservabilità	D5.P1, D5.P2, D5.P3, D5.P4, D5.P5
D6: CI/CD & DevEx	D6.P1, D6.P2, D6.P3
D7: Packaging & Deployment	D7.P1, D7.P2, D7.P3

un'altra. Comprenderle nell'ordine in cui si vincolano a vicenda è il modo più diretto per capire l'intera architettura.

3.1.1 Agnosticismo Applicativo (D1.P1)

Qualsiasi Application Programming Interface (API) REST documentata da una specifica OpenAPI è testabile senza modificare una sola riga di codice. Non si tratta di una promessa di portabilità generica, ma di un vincolo strutturale imposto fin dall'origine: APIGuard Assurance non contiene nessun riferimento hardcoded a path, strutture dati o comportamenti di un'applicazione specifica. Tutta la conoscenza del target viene derivata a runtime da due sole sorgenti: il file `config.yaml`, che raccoglie i parametri operativi quali Uniform Resource Locator (URL), credenziali e soglie di esecuzione, e la specifica OpenAPI del target, che fornisce la topologia completa degli endpoint, gli schemi dei payload e i tipi di risposta attesi.

La conseguenza operativa di questo vincolo si manifesta nel comportamento osservabile dei test. Un test che interroga la superficie di attacco derivata dalla specifica e non individua endpoint pertinenti alle proprie condizioni di esecuzione restituisce uno stato `SKIP`, non un errore: lo `SKIP` è la conseguenza prevista di uno scope legittimamente vuoto, distinta dall'`ERROR` che segnalerebbe una condizione imprevista nell'infrastruttura del tool stesso. Quella distinzione semantica non è un dettaglio implementativo; è la forma in cui l'agnosticismo si manifesta nel comportamento osservabile del sistema.

Il discrimine tra uno script e un contributo metodologico risiede esattamente qui. Uno strumento che funziona su un solo target, con path e credenziali scritte nel codice,

è uno script di automazione; uno che deriva tutta la propria conoscenza dal contratto formale del target, senza assunzioni applicazione-specifiche, è un framework riutilizzabile su qualsiasi API REST documentata. Questa proprietà è la principale giustificazione accademica del lavoro.

Il costo accettato è la dipendenza da una specifica valida. Un tool che non può verificare la correttezza del proprio oracolo è un tool che può produrre assessment parziali in presenza di specifiche errate o intenzionalmente incomplete, senza diagnosticarne l'incoerenza. Questa tensione costituisce il limite strutturale discusso nella Sezione ?? e si qualifica come problema aperto nel testing contract-driven, non come difetto correggibile con intervento locale. Stabilire dove si trovano i confini del sistema è parte integrante della sua descrizione architettuale.

3.1.2 La Specifica OpenAPI come Unico Oracolo Formale (D3.P2)

L'agnosticismo applicativo stabilisce la sorgente della conoscenza del target; D3.P2 determina come quella conoscenza viene impiegata. La specifica OpenAPI non è documentazione destinata al tester umano: è il vincolo contrattuale che guida l'intera pipeline. Non descrive il target, lo vincola, e quella differenza è decisiva per la classe di vulnerabilità che il tool è in grado di rilevare.

Un fuzzer tradizionale che genera payload casuali affronta un ostacolo strutturale già prima di raggiungere la logica applicativa: la maggioranza delle richieste viene rifiutata con HTTP 400 dal layer di validazione dell'input, perché i tipi, i formati e i vincoli enumerati nello schema non sono rispettati. Questo è il *combinatorial wall* discusso nel Capitolo 2: lo spazio di tutti i payload sintatticamente possibili è vastissimo, ma lo spazio dei payload strutturalmente validi rispetto al contratto è definito e navigabile. Usare la specifica come oracolo formale abbatte quella barriera: ogni richiesta è costruita nel rispetto dei vincoli dichiarati, spostando il punto di osservazione dalla sintassi alla semantica.

Ne derivano due capacità operativamente distinte rispetto al fuzzing cieco. In primo luogo, la superficie di attacco viene delimitata con precisione: gli endpoint documentati dalla specifica costituiscono l'insieme di ciò che il contratto dichiara raggiungibile, mentre qualsiasi endpoint risponda senza essere incluso in quella lista è, per definizione, una Shadow API. In secondo luogo, le risposte ricevute vengono valutate rispetto a ciò che il contratto prevede per quella richiesta, non rispetto a euristiche generiche: un campo sensibile restituito in risposta a una richiesta autenticata è un finding solo se lo schema di risposta contrattuale non lo dichiarava atteso.

Anche questa proprietà porta con sé un costo che vale la pena enunciare con precisione. Più il sistema si affida alla specifica come oracolo, più il suo comportamento dipende

dalla correttezza di quell'oracolo. Una specifica obsoleta o deliberatamente incompleta produce un assessment parziale senza che il sistema possa rilevarne l'incoerenza: si tratta del paradosso del Ground Truth, discusso nella Sezione ?? come limite strutturale del paradigma contract-driven e non come anomalia specifica di questa implementazione.

3.1.3 Config-Driven Development (D3.P1)

Stabilito che la conoscenza del target risiede nella specifica OpenAPI, rimane da definire il meccanismo che governa l'esecuzione. D3.P1 risponde a questa questione in modo perentorio: ogni parametro di tuning operativo risiede nel file `config.yaml`. Il codice sorgente non contiene literal decisionali hardcoded. Soglie di timeout, credenziali, URL del gateway, strategie di test abilitate, configurazione del path seed: tutto ciò che può variare tra un deployment e l'altro è dichiarato esplicitamente nel file di configurazione e validato a runtime contro uno schema Pydantic prima che la pipeline si avvii.

Il vantaggio non è solo organizzativo. Separare la configurazione dal codice elimina la classe di bug in cui il comportamento del tool cambia in assenza di qualsiasi modifica al codice stesso, perché qualcuno ha alterato un parametro implicito. Con D3.P1 questa classe di bug non può esistere per costruzione: se il comportamento è cambiato, è cambiata la configurazione, e quella modifica è tracciabile, versionabile e auditabile come qualsiasi altro artefatto del progetto.

La relazione con la proprietà successiva è di tipo causale, non di semplice coesistenza. D3.P1 è il prerequisito strutturale di D3.P4: senza la centralizzazione di ogni parametro decisionale in un file di configurazione immutabile durante il run, la riproducibilità non è garantibile. Un sistema che legge valori da ambienti variabili non dichiarati, da file di stato impliciti o da sorgenti runtime non controllate non può produrre lo stesso risultato a ogni esecuzione, indipendentemente dalla qualità della logica di test.

3.1.4 Riproducibilità Deterministica (D3.P4)

Un assessment di sicurezza non riproducibile non è un assessment: è un'osservazione irripetibile. Lo stesso target, nella stessa configurazione, interrogato con lo stesso `config.yaml`, deve produrre risultati deterministicamente identici a ogni esecuzione. Questa non è una proprietà di comodità operativa, ma il presupposto epistemico che conferisce valore dimostrativo ai risultati del Capitolo 5.

Il determinismo è ottenuto attraverso tre meccanismi distinti che si completano a vicenda. Il primo è il config-driven development già descritto in D3.P1: se tutti i parametri decisionali sono fissi, la pipeline non ha sorgenti di variazione endogene. Il secondo è l'ordinamento deterministico del registro dei test: il `TestRegistry` ordina i test per

`test_id` lessicografico prima di passarli al scheduler, garantendo che l'insieme di partenza del grafo sia sempre lo stesso. Il terzo è la struttura del Directed Acyclic Graph (DAG) stesso, trattata in Sezione 3.2.2: un grafo aciclico orientato con ordinamento topologico deterministico produce sempre la stessa sequenza di batch, indipendentemente dall'ordine in cui i test vengono scoperti a runtime.

La garanzia di riproducibilità non implica che l'output sia byte-identico in ogni sua componente. Timestamp e identificatori Universally Unique Identifier (UUID) dei singoli record vengono esclusi dal confronto; ciò che è invariante sono i contatori PASS, FAIL e SKIP, la distribuzione dei finding per dominio, e la sequenza di batch prodotta dal DAG. La validazione empirica di questa proprietà, condotta su due run indipendenti con un delta sul wall-clock di 0,32%, è discussa nella Sezione ??.

Queste quattro proprietà, considerate insieme, rispondono alla domanda più generale che un lavoro di ricerca deve saper porre a se stesso: perché questo sistema, in questa forma, è un contributo e non un artefatto? La risposta è che un tool applicazione-agnostico (D1.P1), guidato da un oracolo formale (D3.P2), controllato da configurazione esplicita (D3.P1) e deterministicamente riproducibile (D3.P4) può essere validato, confrontato e integrato in processi automatici, laddove un tool manuale, opaco e target-specifico non può. Nella Sezione 3.2 si mostrerà come questi principi si traducano in scelte strutturali concrete.

3.2 Architettura dell'Assessment Engine

I principi enunciati nella sezione precedente rispondono alla domanda del perché. Questa sezione risponde alla domanda del come: come quei principi si traducono in struttura, come i componenti si relazionano tra loro, e perché quella struttura è stata scelta rispetto alle alternative disponibili. Ogni proprietà discussa qui risponde a una frizione architetturale specifica; presentarle in isolamento, senza nominare il problema che ciascuna risolve, ridurrebbe la descrizione a un inventario di decisioni opache.

3.2.1 Flusso di Dipendenze Monodirezionale (D1.P2)

Prima ancora di descrivere i singoli componenti, occorre descrivere la regola che ne governa le relazioni. Il grafo delle dipendenze tra moduli è strettamente aciclico e orientato in una sola direzione: `core/` è importato da `connectors/`, che è importato da `tests/`, che è importato da `engine.py`. Nessun modulo può importare da un livello superiore nella gerarchia. In termini operativi: modificare un test non può rompere `core/`; aggiungere un connector non può influenzare `engine.py`; l'intera base di `core/` può essere testa-

ta unitariamente senza istanziare alcun HyperText Transfer Protocol (HTTP) client né alcun test di sicurezza.

Questa regola non emerge da preferenze stilistiche, ma dal problema che risolve. In assenza di un vincolo esplicito sulla direzione delle dipendenze, la pressione evolutiva naturale del codice tende verso cicli: un test che trova conveniente importare un'utility da un altro test, un connector che ha bisogno di un modello definito in `engine.py`. Ogni ciclo rende un modulo impossibile da isolare, e un modulo non isolabile non è testabile né sostituibile. Il vincolo monodirezionale è verificabile staticamente con un singolo comando (`grep -r "from src.tests" src/core/`: zero risultati) e viene dichiarato nella docstring di apertura di ogni modulo in `core/`, rendendolo parte del contratto pubblico del package, non una convenzione implicita.

Ne deriva, come conseguenza diretta, la praticabilità del plugin system descritto come traiettoria futura nella Sezione ??: un modulo di test contribuito da terze parti può essere aggiunto al `sys.path` e viene scoperto automaticamente senza che nessuna modifica al `core/` sia necessaria, perché il `core/` non sa nulla dei test.

3.2.2 Scheduling Basato su DAG e Orchestrazione Deterministica (D2.P2)

L'ordine in cui i test vengono eseguiti non è arbitrario, né è codificato a mano in un file di configurazione. Viene calcolato a partire dalle dipendenze che ogni test dichiara esplicitamente tramite la `ClassVar depends_on`. Il `DAGScheduler` in `src/core/dag.py` raccoglie queste dichiarazioni, costruisce un DAG orientato, e delega la risoluzione dell'ordine topologico a `graphlib.TopologicalSorter`, il modulo della libreria standard Python introdotto nella versione 3.9. L'output è una lista ordinata di `ScheduledBatch`: oggetti frozen che raggruppano i test il cui insieme di dipendenze è già stato soddisfatto. I batch sono sequenziali tra loro; all'interno di ogni batch, i test sono privi di dipendenze reciproche.

Il problema che questa struttura risolve non è solo di ordinamento, ma di correttezza semantica. Un test che verifica la validità crittografica di un token JSON Web Token (JWT) (test 1.2) presuppone che il meccanismo di autenticazione funzioni (test 1.1): se 1.1 non viene eseguito prima di 1.2, il token non è disponibile nel `TestContext` e 1.2 fallirebbe per ragione sbagliata. Dichiarare `depends_on=["1.1"]` su test 1.2 sposta questa preconditione dallo stato implicito al contratto esplicito, verificato dal DAG prima che la pipeline entri in esecuzione.

Due meccanismi di sicurezza complementano l'ordinamento. Il primo è la rilevazione statica dei cicli: se il grafo delle dipendenze contiene un ciclo, `DAGCycleError` viene sollevata in Phase 4, prima che un singolo test venga eseguito. Il secondo è la stall detection:

se `TopologicalSorter` segnala `is_active() = True` ma `get_ready()` non restituisce nodi, il sistema registra in formato strutturato l'elenco esatto dei test non estratti e interrompe il drain, garantendo che l'operatore possa identificare immediatamente quale dichiarazione `depends_on` ispezionare.

Il legame con la riproducibilità descritta in D3.P4 è causale, non accidentale. I `test_id` all'interno di ogni batch vengono ordinati lessicograficamente prima di essere restituiti, perché `graphlib` non garantisce l'ordine all'interno di un livello topologico. Quella singola riga di codice, `sorted(sorter.get_ready())`, è il punto in cui il non-determinismo del grafo viene convertito in sequenza riproducibile. Come verificato nella Sezione ??, 16 test in Phase A e 2 in Phase B producono la stessa sequenza a ogni run.

La struttura a batch ha infine una conseguenza per il futuro: ogni batch è già concettualmente parallelizzabile. L'aggiunta di un `ThreadPoolExecutor` in Phase 5 dell'engine è una modifica localizzata che non richiede modifiche al design dei test, purché le questioni di thread-safety su `TestContext` ed `EvidenceStore` vengano risolte con analisi formale. Questo limite è documentato e noto; la decisione di non parallelizzare nella versione attuale è una scelta esplicita, non una dimenticanza.

3.2.3 Split State e Immutabilità (D1.P3)

Durante l'esecuzione sequenziale dei test, due categorie di informazione coesistono nel processo: ciò che si sapeva del target prima che la pipeline iniziasse, e ciò che si scopre o si produce durante l'esecuzione. Trattarle con lo stesso oggetto mutabile è la ricetta per una classe di bug difficile da diagnosticare: un test che scrive accidentalmente sulla rappresentazione del target altera la vista condivisa da cui ogni verdetto dipende, senza che il sistema possa rilevarlo.

La soluzione è la separazione netta in due oggetti con caratteristiche opposte. Il `TargetContext`, configurato con `model_config = {"frozen": True}` in Pydantic v2, incapsula tutto ciò che si conosce del target prima dell'esecuzione: la specifica OpenAPI dereferenziata come `AttackSurface`, le credenziali, i parametri di tuning, l'adapter del gateway iniettato in Phase 3. Qualsiasi tentativo di modifica su un campo del `TargetContext` solleva `ValidationError` al momento dell'assegnazione, non in un punto remoto della pipeline: l'errore è rilevato dove si verifica, non dove il suo effetto diventa osservabile. Il `TestContext`, al contrario, è mutabile e raccoglie ciò che la pipeline produce: i token acquisiti, le risorse create sul target che dovranno essere rimosse al teardown, i dati condivisi tra test dipendenti.

L'accesso allo stato mutabile avviene attraverso tre canali tipizzati distinti, non attraverso un dizionario libero. Il canale token (`set_token`, `get_token`) è indicizzato per costante di ruolo (`ROLE_ADMIN`, `ROLE_USER_A`, `ROLE_USER_B`), eliminando le magic string.

Il canale teardown (`register_resource_for_teardown`, `drain_resources`) mantiene un registro Last In, First Out (LIFO) delle risorse create durante il run, garantendo che il cleanup segua l'ordine inverso della creazione, come discusso nella Sezione 4.5.3. Il canale shared data (`set_shared`, `get_shared`) permette a test in fase B o C di consumare dati prodotti da test in fase A senza introdurre dipendenze di importazione tra moduli: la convenzione di nomenclatura "`{test_id}.{data_name}`" rende sempre esplicito il test che ha prodotto il dato.

Il costo accettato è la rigidità del `TargetContext`. Qualsiasi informazione scoperta a runtime, inclusi gli endpoint presenti sul target ma non dichiarati nella specifica, appartiene allo stato dell'assessment e non allo stato del target: non può quindi essere scritta sul `TargetContext`, ma trova il suo posto nel canale shared del `TestContext`. Questo confine non è sempre intuitivo, ma la sua esplicitezza è la garanzia che rende la riproducibilità di D3.P4 dimostrabile senza ambiguità.

3.2.4 Gateway-Agnostic Adapter (D1.P6)

L'agnosticismo applicativo dichiarato in D1.P1 riguarda la conoscenza del target applicativo: la specifica OpenAPI e il `config.yaml` come sole sorgenti. D1.P6 estende lo stesso principio a un livello distinto: lo strato di controllo del gateway. Alcune garanzie di sicurezza non sono osservabili dall'esterno del sistema, cioè interrogando gli endpoint esposti ai client. La verifica che un timeout sia configurato sui servizi upstream, che il plugin di circuit breaking sia attivo, che le credenziali di servizio non siano hardcoded nelle variabili d'ambiente del gateway: queste informazioni risiedono nel piano di configurazione amministrativo del gateway, non nella superficie HTTP che il client vede. Accedervi richiede un meccanismo separato dagli HTTP probe ordinari.

Il `BaseGatewayAdapter`, definito come classe base astratta in `src/core/gateway/base.py`, espone un'interfaccia uniforme e read-only: `check_connectivity()`, `get_routes()`, `get_plugins()`, `get_services()`, `get_upstreams()`, `get_plugin_by_name()` e `get_status()`. I test `WHITE_BOX` accedono al gateway esclusivamente tramite questa interfaccia, senza importare né referenziare il tipo concreto sottostante. L'implementazione concreta, `KongGatewayAdapter`, interroga la Admin API di Kong DB-less tramite chiamate HTTP dedicate, distinte dal traffico verso il target applicativo: la separazione dei due canali è una scelta deliberata, documentata nel modulo, perché le chiamate amministrative non devono apparire nell'`EvidenceStore` né essere soggette alle stesse policy di retry del traffico di test.

La selezione del tipo concreto avviene a runtime: `engine.py` legge il campo `gateway_adapter` dal `config.yaml`, istanzia il tipo corrispondente, verifica la raggiungibilità tramite `check_connectivity()`, e inietta l'istanza in `TargetContext.gateway`. Se la Admin API non è configurata, il campo rimane `None` e tutti i test `WHITE_BOX` transitano a `SKIP` tramite l'helper

`_requires_admin_api()`, senza errori a cascata: la degradazione è controllata, non silenziosa.

La conseguenza diretta per l'estensibilità è precisa. Aggiungere supporto per Traefik o nginx Plus richiede una nuova sottoclasse di `BaseGatewayAdapter` e una riga nel dispatcher di Phase 3 in `engine.py`. Nessuno dei test che usano `target.gateway.*` deve essere modificato, perché la loro logica opera sull'interfaccia astratta, non sull'implementazione concreta. Il limite intrinseco di questa astrazione, vale a dire la tensione tra la generalità dell'interfaccia e la profondità delle verifiche vendor-specific che essa non può esprimere, costituisce uno dei tre limiti strutturali discussi nella Sezione ??.

3.2.5 Streaming Evidence Store (D4.P1)

Ogni finding prodotto da un test deve essere corredato da evidenza HTTP dimostabile: request inviata, response ricevuta, timestamp, identificatori di traccia. Questa non è una funzionalità opzionale per la presentazione del report; è il requisito di non-repudiabilità che conferisce valore legale e tecnico a un audit trail. Un finding senza evidenza è un'asserzione.

Il design originale dell'`EvidenceStore` utilizzava un buffer a capacità fissa (`deque(maxlen=100)`): i record più vecchi venivano evicti silenziosamente quando il buffer si riempiva, generando `Finding.evidence_ref` che puntavano a record inesistenti. L'audit trail risultava rotto senza che il sistema producesse nessun errore visibile. Il difetto era strutturale, non correggibile con un valore di `maxlen` più grande: qualsiasi limite fisso avrebbe trasformato la capacità in un parametro di tuning il cui valore corretto dipende da ciò che si vuole misurare, informazione non disponibile a priori.

La risposta adottata elimina la classe di bug per costruzione. L'`EvidenceStore` scrive ogni record come oggetto JavaScript Object Notation (JSON) su file JSON Lines (JSONL) con flush immediato, senza buffering in memoria. Il ciclo di vita è articolato in quattro operazioni: `begin_test()` apre il file temporaneo dedicato al test corrente; la scrittura per-record avviene con flush su disco a ogni record; `end_test()` chiude il file; `merge_and_finalize()` in Phase 7 concatena tutti i file temporanei nel file `evidence.json` definitivo. Non esiste un limite superiore al numero di record per test: test con superfici di attacco ampie, come quelli sul Domain-0 che iterano su ogni endpoint documentato nella specifica, producono tanti record quanti ne sono necessari.

La relazione con D3.P5, proprietà discussa nella sottosezione seguente, è di tipo prerequisito. L'`EvidenceStore` scrive su file locali nella directory `outputs/`, senza dipendenze verso database o storage remoto. Questo vincolo non è accidentale: è la condizione necessaria perché l'intero assessment sia eseguibile in ambienti senza infrastruttura ester-

na, inclusi i runner Continuous Integration / Continuous Deployment (CI/CD) privi di servizi accessori.

3.2.6 Assenza di Dipendenze di Stato Esterno (D3.P5)

Un tool che richiede un database, un message broker o una cache esterna per funzionare sposta il problema della sua affidabilità fuori dai propri confini. L'exit code restituito al termine dell'esecuzione riflette lo stato dell'assessment solo se tutti i servizi da cui dipende erano disponibili e corretti: verificarlo richiede readiness check preventivi che complicano il deployment e rendono il comportamento del tool difficile da ragionare in isolamento.

APIGuard Assurance non ha dipendenze di stato esterno a runtime. Tutto lo stato mutabile del processo risiede in memoria (`TargetContext`, `TestContext`) o in file locali scritti nella directory `outputs/`. Le uniche connessioni di rete aperte durante un run sono quelle verso il target applicativo tramite `SecurityClient` e, opzionalmente, verso la Admin API del gateway tramite `KongGatewayAdapter`. Entrambe sono connessioni di test verso il sistema da valutare, non dipendenze infrastrutturali del tool stesso.

Il vincolo è verificabile per ispezione di `pyproject.toml`: la sezione delle dipendenze non include nessun client di database, nessun client Redis, nessun Software Development Kit (SDK) di storage remoto. Il deployment ridotto al minimo, cioè `pip install più config.yaml`, è la conseguenza diretta di questa scelta. Ed è anche il presupposto che rende affidabile D6.P1: i semantic exit codes discussi nella Sezione ?? possono essere considerati fedeli allo stato dell'assessment perché non esiste alcun servizio accessorio il cui failure silenzioso potrebbe alterarli.

3.3 Domini di Sicurezza e Box Gradient Applicato

Le proprietà architetturali descritte nelle sezioni precedenti definiscono come il sistema è costruito. Questa sezione definisce cosa il sistema misura e secondo quale logica organizza le proprie misure. È il punto di convergenza tra la tassonomia delle minacce discussa nel Capitolo 2, dove le categorie di vulnerabilità sono state analizzate in quanto problema, e la struttura operativa del tool, dove quelle stesse categorie diventano il perimetro di garanzie che un assessment sistematico è tenuto a coprire.

3.3.1 Tassonomia degli Otto Domini di Assurance

La superficie di sicurezza di un'API esposta tramite gateway non è omogenea. Alcune garanzie sono verificabili senza nessuna conoscenza del sistema, interrogando semplicemente gli endpoint esposti senza credenziali; altre richiedono uno stato autenticato; altre ancora diventano accessibili solo attraverso il piano di configurazione amministrativo del gateway. Trattare queste garanzie come una lista piatta, senza riconoscere la struttura di precondizioni che le lega, è la premessa per assessment incompleti: non perché manchino i test, ma perché vengono eseguiti nell'ordine sbagliato o con il livello di privilegio sbagliato.

La tassonomia adottata organizza le garanzie di APIGuard Assurance in otto domini, ciascuno corrispondente a una categoria semantica distinta di rischio. La Tabella 3.2 li riporta con il loro titolo e l'orizzonte delle garanzie che ciascuno racchiude.

Tabella 3.2: I otto Domini di Assurance di APIGuard Assurance e il loro perimetro di garanzie. Domain-5 è architetturalmente predisposto ma implementato nella Milestone 2.

Dominio	Titolo	Perimetro di garanzie
D0	API Discovery	Inventario degli endpoint esposti; rilevazione di Shadow API non documentate nella specifica.
D1	Identity & Authentication	Meccanismi di riconoscimento del chiamante; robustezza del trasporto delle credenziali; ciclo di vita e revoca dei JWT.
D2	Authorization	Logiche di controllo degli accessi; difetti di segregazione orizzontale e verticale (Role-Based Access Control (RBAC), Broken Object Level Authorization (BOLA)).
D3	Data Integrity	Coerenza strutturale dei payload; implementazione delle firme crittografiche (HMAC).
D4	Availability & Resilience	Difese contro l'esaurimento delle risorse; audit di circuit breaker e soglie di rate limiting.
D5	Observability	Qualità e tempestività della telemetria generata dal sistema. (<i>Milestone 2</i>)
D6	Configuration & Hardening	Indurimento dell'infrastruttura di gateway; policy di routing; header di sicurezza HTTP.
D7	Business Logic & Sensitive Flows	Falle semantiche complesse: elusione dei vincoli di processo, Server-Side Request Forgery (SSRF), race condition su operazioni critiche.

La sequenza non è arbitraria. Rispecchia una gerarchia logica di precondizioni: non ha senso verificare l'autorizzazione (D2) se non si è prima verificato che l'autenticazione

funzioni (D1), così come non ha senso verificare l'autenticazione se non si conosce la superficie di attacco (D0). La gerarchia non è una convenzione organizzativa del codice: è una catena logica di assunzioni di fiducia. Un assessment che verifichi D2 senza avere certezze su D1 raccoglie evidenza su un sistema il cui comportamento in caso di autenticazione compromessa è ignoto; quella evidenza non è affidabile.

Questa struttura ha una conseguenza diretta sulla topologia del DAG discussa nella Sezione 3.2.2. I test che operano sui domini logicamente dipendenti dichiarano le proprie precondizioni tramite **depends_on**: test 1.4 (revoca token, D1) dipende da test 1.1 (autenticazione richiesta, D1); test 2.1 (autorizzazione RBAC, D2) dipende anch'esso da test 1.1, perché senza token validi per due ruoli distinti non è possibile distinguere un rifiuto di autorizzazione da un rifiuto di autenticazione. La gerarchia dei domini non si traduce in un ordine totale rigido tra tutti i test, ma in un insieme di dipendenze dichiarate che il DAG soddisfa automaticamente.

Domain-5 occupa una posizione particolare nella tassonomia: il dominio è architetturealmente predisposto nella struttura della codebase, ma i test corrispondenti sono pianificati per la Milestone 2. La numerazione è riservata intenzionalmente per preservare la coerenza della sequenza nelle versioni successive.

3.3.2 Il Box Gradient come Mapping dalle Precondizioni al Privilegio (D3.P3)

La classificazione Black/Grey/White Box del testing è nota nella letteratura come distinzione tra gradi di visibilità del sistema sotto test. In APIGuard Assurance quella classificazione non viene applicata come categoria analitica astratta: viene derivata dalle precondizioni concrete che ogni garanzia impone al tester prima ancora che il test possa iniziare. La differenza è sostanziale. Una tassonomia applicata retroattivamente etichetta i test dopo che sono stati scritti; una mappatura derivata dalle precondizioni determina quale postura adottare prima di scrivere il test, perché alcune garanzie sono fisicamente inaccessibili senza un livello di privilegio specifico.

Il mapping adottato, formalizzato come proprietà D3.P3 e sintetizzato nella Tabella 3.3, correla il livello di priorità del test alla strategia operativa che le sue precondizioni impongono.

Assegnare a ogni test un livello di visibilità non è dunque una scelta organizzativa, ma una conseguenza diretta di ciò che si vuole misurare. Un controllo perimetrale, ovvero la verifica che il gateway rifiuti richieste non autenticate, non presuppone nessuna conoscenza dell'infrastruttura interna: il risultato atteso è lo stesso indipendentemente da chi effettua la richiesta. Simulare un attaccante esterno senza credenziali è l'unico

Tabella 3.3: Mapping tra priorità di test, strategia operativa e prerequisiti tipici in API-Guard Assurance (D3.P3). La colonna Prerequisiti indica le risorse che l'operatore deve fornire prima di avviare i test di quel livello. Il mapping è una raccomandazione strutturale, non un vincolo imposto dal framework: deviazioni architetturalmente giustificate esistono nel codebase.

Priorità	Strategia	Prerequisiti	Razionale
P0 (Critico)	BLACK_BOX	Nessuno (solo accesso di rete)	Controlli perimetrali che il gateway applica a qualsiasi interlocutore, autenticato o no.
P1 (Alto)	GREY_BOX	Token per ≥ 2 ruoli distinti	Garanzie che presuppongono stato autenticato; inaccessibili senza credenziali valide.
P2 (Medio)	GREY_BOX / WHITE_BOX	Variabile per garanzia	Mix di test empirici e configuration audit; specificato per ogni garanzia.
P3 (Basso)	WHITE_BOX	Accesso Admin API del gateway	Configurazione statica verificabile per ispezione diretta, non tramite probe empirici.

modo per ottenere evidenza non contaminata da privilegi impliciti, ed è per questo che i test P0 adottano la strategia BLACK_BOX.

Un controllo di autorizzazione RBAC, al contrario, richiede che il sistema si trovi in uno stato autenticato con almeno due identità di ruolo distinto, un amministratore e un utente ordinario, ad esempio. Senza quello stato il test non raggiunge nemmeno la logica che intende sondare: il gateway respinge la richiesta al livello di autenticazione, prima ancora che la logica di autorizzazione venga valutata, e il risultato osservato è indistinguibile da un corretto rifiuto di autenticazione. Di conseguenza, i test P1 e P2 che operano su D2 adottano la strategia GREY_BOX, con token validi per ruoli distinti forniti dall'operatore prima dell'avvio.

I test WHITE_BOX di priorità P3 occupano una posizione metodologicamente distinta. Alcune garanzie sono più economicamente verificabili tramite ispezione diretta della configurazione del gateway che tramite probe empirici verso gli endpoint: la versione Transport Layer Security (TLS) supportata, la presenza e la parametrizzazione dei plugin di sicurezza, l'assenza di credenziali hardcoded nelle variabili d'ambiente. Un test empirico che cerchi di inferire questi parametri dall'esterno richiederebbe attacchi elaborati con esito incerto; un audit della configurazione li verifica in modo deterministico e riproducibile. È per questo che i test P3 delegano la raccolta dei dati al `BaseGatewayAdapter` descritto nella Sezione 3.2.4, anziché effettuare probe sull'interfaccia pubblica.

Va rilevato che il mapping tra priorità e strategia è una raccomandazione strutturale,

non un vincolo rigido imposto dal framework. Il codebase contiene deviazioni architetturalmente giustificate: test 7.2 (SSRF, Domain-7) ha priorità P0 ma strategia `GREY_BOX`, perché la verifica di un endpoint che accetta URL controllabili dall'utente per effettuare richieste lato server richiede che l'endpoint sia raggiungibile con una richiesta autenticata. Il framework non vieta questa deviazione: la `ClassVar strategy` appartiene al test, non al dominio, e la sua correttezza è una responsabilità dello sviluppatore del test, non del registro.

Ne deriva che il gradiente Black/Grey/White Box, applicato in questa forma, non è una tassonomia teorica importata dalla letteratura e applicata retroattivamente alla struttura del tool. È il prodotto delle precondizioni informative e operative che ciascuna garanzia di sicurezza impone al tester: la tecnica segue il requisito, non viceversa. Questa distinzione, apparentemente sottile, ha conseguenze pratiche rilevanti per l'usabilità del framework in contesti industriali, dove la disponibilità delle credenziali e dell'accesso amministrativo varia da deployment a deployment e non può essere assunta come uniforme. La Sezione ?? discute come questa variabilità di privilegio si integri con le pipeline CI/CD senza richiedere configurazioni fisse.

Capitolo 4

Implementazione

Le 39 proprietà architetture formalizzate nel Capitolo 3 trovano in questo capitolo la loro traduzione in codice Python. Non tutte ricevono lo stesso grado di trattamento: alcune strutturano interi componenti e vengono discusse in sezioni dedicate; altre emergono come conseguenza naturale delle scelte descritte e vengono citate nel contesto; altre ancora, di natura prettamente implementativa, compaiono nella tabella tassonomica del Capitolo 3 come riferimento. In ogni caso nessuna proprietà viene ignorata: il lettore che vuole rintracciare la realizzazione di una proprietà specifica può usare quella tabella come indice e poi cercare il locus nel codice indicato.

La distinzione rispetto al capitolo precedente è di livello di astrazione. Il Capitolo 3 descrive cosa fanno i componenti e perché sono stati progettati in quella forma; questo capitolo descrive come lo fanno, nominando classi, metodi e moduli specifici. I due livelli si richiamano con riferimenti espliciti: dove un componente è stato introdotto in termini architetture nel Capitolo 3, questo capitolo aggiunge il dettaglio implementativo senza ripetere la motivazione.

4.1 La Pipeline di Esecuzione a Sette Fasi e il Core Engine

L'intero assessment si svolge in sette fasi sequenziali, orchestrate da `engine.py`. La struttura a fasi non è una convenzione organizzativa: riflette la dipendenza logica tra le operazioni e determina la semantica degli errori. Le fasi 1, 2 e 4 sono bloccanti, nel senso che un errore al loro interno interrompe lo startup prima che un singolo test venga eseguito; le fasi 5, 6 e 7 non bloccano mai, e i loro fallimenti vengono registrati nell'evidenza senza interrompere la pipeline. La Tabella 4.1 riporta il mapping completo tra fase, modulo responsabile e comportamento in caso di errore.

Tabella 4.1: Le sette fasi della pipeline di assessment di APIGuard Assurance. Le fasi 1, 2 e 4 sono bloccanti: un errore interrompe lo startup prima dell'esecuzione dei test. Le fasi 5, 6 e 7 sono non-bloccanti: i fallimenti vengono registrati e l'esecuzione prosegue.

Fase	Nome	Modulo	Comportamento in caso di errore
1	Inizializzazione	<code>config/loader.py</code>	<code>ConfigurationError</code> — blocca lo startup
2	OpenAPI Discovery	<code>discovery/openapi.py</code>	<code>OpenAPILoadError</code> — blocca lo startup
3	Costruzione Contesti	<code>engine.py</code>	Nessuna eccezione attesa; <code>TargetContext</code> , <code>TestContext</code> e <code>EvidenceStore</code> vengono istanziati
4	Test Discovery e DAG	<code>tests/registry.py</code> , <code>core/dag.py</code>	<code>DAGCycleError</code> — blocca lo startup
5	Esecuzione	<code>engine.py</code>	Eccezioni per-test catturate internamente; il test produce <code>TestResult(ERROR)</code> e la pipeline prosegue
6	Teardown	<code>core/context.py</code>	<code>TeardownError</code> registrato come <code>WARNING</code> ; non invalida i risultati
7	Report	<code>report/renderer.py</code>	Output su <code>evidence.json</code> e <code>assessment_report.html</code>

Il ruolo di `engine.py` in questa struttura è di orchestratore puro. Il file esegue le fasi nell'ordine corretto, passa i contesti da una fase all'altra, e non contiene nessuna logica di dominio: nessuna regola di sicurezza, nessun criterio di valutazione, nessuna conoscenza del formato della specifica OpenAPI. Questa separazione non è estetica. Un orchestratore che contiene logica di dominio diventa il punto in cui si concentrano le dipendenze, rendendo difficile testare i componenti in isolamento e impossibile sostituire una fase senza rischiare di rompere l'altra. La coerenza di `engine.py` con il principio del flusso monodirezionale di D1.P2, discusso nella Sezione 3.2.1, è verificabile per ispezione diretta: il file importa da `core/`, `discovery/`, `tests/` e `report/`, ma nessuno di questi moduli importa da `engine.py`.

4.1.1 Fail-Safe Error Isolation (D4.P3)

Il contratto che regola il rapporto tra `engine.py` e i test individuali è espresso da un'assenza: in `engine.py` non esiste nessun `try/except` attorno alla chiamata a `test.execute()`. Non si tratta di una dimenticanza. È la dichiarazione esplicita che la responsabilità di gestire le proprie eccezioni appartiene al test, non all'orchestratore.

Il contratto imposto da `BaseTest.execute()` stabilisce che il metodo deve sempre restituire un `TestResult` e non può mai propagare eccezioni verso l'esterno. La realizzazione concreta è un blocco `try/except Exception` all'interno di `execute()`, che in caso di eccezione imprevista costruisce e restituisce un `TestResult(status=ERROR, message=str(e))` con il traceback troncato a 500 caratteri tramite l'helper `_make_error()`. La pipeline riceve sempre un `TestResult` valido, indipendentemente da ciò che è successo internamente al test: nessun test può bloccare l'esecuzione degli altri.

La conseguenza operativa è misurabile. Un test che va in eccezione per un bug interno, per una risposta HTTP inattesa, o per un timeout non gestito produce un `TestResult(ERROR)` che viene registrato nell'evidenza con il suo traceback, e la pipeline prosegue con il test successivo nel batch. L'errore è visibile e diagnosticabile; la solidità della pipeline non dipende dalla correttezza di ogni singolo test.

4.1.2 Gerarchia delle Eccezioni con Phase Mapping (D4.P4)

Il meccanismo che rende possibile la distinzione tra errori bloccanti e non-bloccanti descritta in Tabella 4.1 è la gerarchia delle eccezioni custom. Tutte le eccezioni del tool sono sottoclassi di `ToolBaseError`, radice comune che permette al codice chiamante di catturare qualsiasi errore del tool con un singolo `except ToolBaseError` o di essere preciso con `except ConfigurationError`. La gerarchia conta 11 classi distribuite in 3 file, ciascuna con un mapping esplicito alla fase della pipeline in cui può essere sollevata. La Tabella 4.2 riporta la classificazione completa.

Due scelte meritano commento esplicito perché riflettono decisioni di design consapevoli, non vincoli tecnici.

La prima è la collocazione di `GatewayAdapterError` in `gateway/base.py` invece che in `exceptions.py`. La scelta segue il principio di locality: l'eccezione è definita nello stesso file dell'interfaccia che la solleva, rendendo il contratto dell'ABC autosufficiente. Un consumer di `BaseGatewayAdapter` trova tutto ciò che gli serve in un unico modulo, senza dover importare da `core/exceptions.py`.

La seconda è l'assenza deliberata di `ExternalToolNotFoundError`. Un tool esterno mancante non è una condizione di errore imprevista: è una condizione operativa at-

Tabella 4.2: Le 11 classi di eccezione custom di APIGuard Assurance, distribuite su 3 file sorgente. Le eccezioni di Phase 1–4 sono fatali e interrompono lo startup; quelle di Phase 5–6 vengono recuperate localmente. Le eccezioni CLI non transitano dall’engine e sono architetturealmente isolate dal loop di assessment.

Classe	Fase	File	Comportamento
<code>ToolBaseError</code>	—	<code>exceptions.py</code>	Radice della gerarchia
<code>ConfigurationError</code>	1	<code>exceptions.py</code>	Fatale — config non valida
<code>OpenAPILoadError</code>	2	<code>exceptions.py</code>	Fatale — spec irraggiungibile o invalida
<code>DAGCycleError</code>	4	<code>exceptions.py</code>	Fatale — ciclo nelle dipendenze
<code>AuthenticationSetupError</code>	helper	<code>exceptions.py</code>	Fatale se le credenziali sono invalide
<code>SecurityClientError</code>	5	<code>exceptions.py</code>	Recuperata — produce <code>TestResult(ERROR)</code>
<code>ExternalToolError</code>	5	<code>exceptions.py</code>	Recuperata — produce <code>TestResult(ERROR)</code>
<code>TeardownError</code>	6	<code>exceptions.py</code>	Recuperata — registrata come <code>WARNING</code>
<code>GatewayAdapterError</code>	5	<code>gateway/base.py</code>	Recuperata — produce <code>TestResult(ERROR)</code>
<code>SeedGeneratorFetchError</code>	CLI	<code>seed_generator.py</code>	Catturata in <code>cli.py</code> , exit code 10
<code>SeedGeneratorParseError</code>	CLI	<code>seed_generator.py</code>	Catturata in <code>cli.py</code> , exit code 10

tesa, gestita a monte durante la Phase 4 tramite `is_available()` nel registry dei connector. Il test che dipende da un tool assente riceve uno stato `SKIP` con motivo esplicito, non un’eccezione. Introdurre `ExternalToolNotFoundError` avrebbe aggiunto una classe senza aggiungere semantica: la condizione è già rappresentata da `TestResult(status=SKIP)`.

4.2 Discovery Dinamico e Risoluzione del Grafo

Prima che un singolo test venga eseguito, l’engine deve sapere quali test esistono e in quale ordine eseguirli. Queste due responsabilità sono delegate a due componenti distinti: il `TestRegistry`, che scopre i test a runtime per ispezione del filesystem, e il `DAGScheduler`, che calcola l’ordine topologico a partire dalle dipendenze dichiarate. La separazione non è casuale: il registry opera su stringhe e classi Python, il scheduler opera su grafi di stringhe. Nessuno dei due conosce la logica interna dei test.

4.2.1 Discovery Dinamico via pkgutil (D2.P1)

Il `TestRegistry` in `src/tests/registry.py` non contiene nessuna lista hardcoded di test. Scopre le classi concrete di test attraverso tre fasi interne, denominate R1, R2 e R3 nella documentazione del modulo.

La Phase R1 è la scansione del filesystem tramite `pkgutil.walk_packages()`, che percorre ricorsivamente le directory `tests/domain_*` e importa tutti i moduli il cui nome rispetta il prefisso `test_`. La convenzione di nomenclatura è tecnica, non organizzativa: `pkgutil` usa il nome del file come discriminante, quindi un file chiamato diversamente non viene scoperto anche se implementa correttamente il contratto. Errori di importazione nei singoli moduli vengono catturati e registrati come `WARNING` strutturato senza interrompere la scansione degli altri.

La Phase R2 estrae le sottoclassi concrete di `BaseTest` dai moduli appena importati tramite `inspect.getmembers()`. Una classe è considerata concreta se non è `BaseTest` stessa, non è astratta, ed è effettivamente una sottoclasse. La Phase R3 applica i filtri: vengono trattenuti solo i test la cui `priority` rientra nelle priorità abilitate e la cui `strategy` rientra nelle strategie abilitate nel `config.yaml`.

L'`ExternalTestRegistry` in `src/external_tests/registry.py` segue la stessa logica con una fase aggiuntiva, denominata R4: dopo il filtraggio, raggruppa i test per `tool_name`, chiama `is_available()` una sola volta per gruppo, e inietta il connector condiviso nei test del gruppo se il tool è disponibile, oppure imposta `_skip_reason_from_registry` su ogni test del gruppo se il tool è assente. Un solo `WARNING` nel log per tool mancante, indipendentemente da quanti test dipendono da quel tool.

La conseguenza per l'estensibilità è diretta. Aggiungere un nuovo test richiede creare un file nella directory di dominio corretta con il nome corretto e implementare il contratto di `BaseTest`: nessun altro file del sistema deve essere modificato per il discovery. Questa proprietà, formalizzata come D2.P1, è il prerequisito tecnico del plugin system menzionato come traiettoria futura nella Sezione ???: un package esterno che aggiunge la propria directory al `sys.path` e rispetta le convenzioni di nomenclatura viene scoperto automaticamente senza modifiche al core.

4.2.2 Scheduling Topologico e DAGScheduler (D2.P2)

Il `DAGScheduler` in `src/core/dag.py` riceve la lista ordinata di `test_id` prodotta dal registry e produce una lista ordinata di `ScheduledBatch`. Come illustrato nella Sezione 3.2.2 a livello architetturale, l'ordinamento è determinato dalle dipendenze dichiarate tramite la `ClassVar depends_on` di ogni test. Qui si descrive come quella logica è implementata.

Il componente centrale è `graphlib.TopologicalSorter` della libreria standard Python 3.9. Il `DAGScheduler` costruisce il grafo passando a `TopologicalSorter` un dizionario che mappa ogni `test_id` all'insieme dei suoi predecessori, chiama `prepare()` per verificare l'assenza di cicli, e poi drena il sorter livello per livello con il pattern `get_ready()` / `done()`. Ogni chiamata a `get_ready()` restituisce i nodi il cui insieme di dipendenze è già stato marcato come completato; `done(*ready_nodes)` sblocca i nodi che dipendono da quelli appena estratti. Ogni livello produce un `ScheduledBatch`.

`ScheduledBatch` è un `dataclass(frozen=True)` con due campi: `batch_index` (intero 0-based che riflette il livello topologico) e `test_ids` (lista di stringhe). L'immutabilità è deliberata: una volta costruito, un batch non deve essere modificato dall'engine durante l'esecuzione.

Il determinismo all'interno di ogni batch, requisito di D3.P4 discusso nella Sezione 3.1.4, viene garantito da una singola riga: `ready_nodes = sorted(sorter.get_ready())`. `graphlib` non garantisce l'ordine dei nodi estratti da un livello topologico; l'ordinamento lessicografico esplicito converte il non-determinismo del grafo in una sequenza riproducibile.

La stall detection gestisce uno scenario documentato ma teoricamente anomalo: `is_active()` restituisce `True` ma `get_ready()` non restituisce nodi, indicando che il sorter è in uno stato irrecuperabile nonostante l'assenza di cicli dichiarati. In questo caso il `DAGScheduler` calcola l'insieme dei `test_id` mai estratti, li registra in un `ERROR` strutturato con il campo `unscheduled_test_ids`, e interrompe il drain. L'operatore dispone dell'elenco esatto dei test da ispezionare senza dover leggere un traceback generico.

4.2.3 Invarianti di `TestResult` a Livello di Tipo (D4.P9)

Ogni test produce un `TestResult`, il modello Pydantic che trasporta lo stato finale della verifica verso il report. Il modello potrebbe in linea teorica essere costruito con combinazioni semanticamente inconsistenti: un `PASS` con findings non vuoti, un `FAIL` senza evidenza, un `SKIP` senza motivazione. Ognuna di queste combinazioni produrrebbe un entry di report formalmente valido ma scientificamente privo di significato.

La soluzione adottata è un `@model_validator(mode="after")` in `src/core/models/results.py` che applica tre invarianti al momento della costruzione dell'oggetto, prima che raggiunga qualsiasi consumer:

1. `status=FAIL` richiede `findings` non vuoto. Un `FAIL` senza evidenza non è un `FAIL`: è un'asserzione.

2. `status=SKIP` richiede `skip_reason` non `None`. Ogni SKIP deve essere documentabile e auditabile.
3. `status=PASS` richiede `findings` vuoto. Un PASS non può coesistere con violazioni dichiarate.

Una violazione di uno degli invarianti solleva `pydantic.ValidationError` nel punto in cui l'helper `_make_pass()`, `_make_fail()`, o `_make_skip()` costruisce l'oggetto, ovvero all'interno del test che ha generato il risultato inconsistente. L'errore è immediatamente localizzato: il traceback punta al test responsabile, non a un punto remoto della pipeline di report.

Il contributo architetturale di questa scelta è la centralizzazione del contratto. Senza il validator, ogni test dovrebbe ricordare autonomamente di non costruire risultati inconsistenti: una convenzione distribuita su 18 implementazioni è una convenzione violabile. Con il validator, la correttezza è enforced da un singolo punto di verifica che non può essere aggirato senza modificare esplicitamente il modello.

4.3 Architettura dei Test Nativi e Contratto `BaseTest`

I test nativi sono le unità di verifica che comunicano direttamente con il target tramite `SecurityClient` e producono evidenza HTTP osservabile. Questa sezione ne descrive la struttura interna: il contratto che ogni test deve rispettare per essere scoperto e orchestrato correttamente, il meccanismo di autenticazione condivisa, le regole di tracciabilità normativa, e la policy con cui i probe HTTP vengono interpretati. La Milestone 1 implementa test per i domini D0, D1, D2, D3, D4, D6 e D7; il Domain-5 (Observability) è architetturalmente predisposto e pianificato per la Milestone 2.

4.3.1 Gerarchia Duale `BaseTest` / `ExternalToolTest` (D1.P4)

Il sistema ospita due categorie di test con contratti distinti, e la distinzione non è organizzativa ma tecnica. I test nativi estendono `BaseTest` in `src/tests/base.py`; i test che integrano tool esterni estendono `ExternalToolTest` in `src/external_tests/base.py`. Le due classi base sono gerarchie parallele indipendenti: `ExternalToolTest` non eredita da `BaseTest`.

La separazione risolve un problema concreto di ereditarietà indesiderata. `BaseTest` espone metodi come `_log_transaction()` e `_probe_endpoint()`, che operano tramite `SecurityClient` per eseguire richieste HTTP contro il target. Un `ExternalToolTest` non usa `SecurityClient`: delega la raccolta dati al proprio connector. Ereditare quei

metodi senza usarli avrebbe ampliato l'interfaccia pubblica di ogni test esterno con funzionalità non applicabili, rendendo il contratto meno preciso e la superficie di errore più ampia.

Dall'esterno, l'engine tratta le due gerarchie in modo quasi identico: entrambe producono un `TestResult`, vengono scoperte dal registry, e vengono schedate dal `DAGScheduler`. L'unica differenza visibile in `engine.py` è un `isinstance` check al momento dell'invocazione, che seleziona la firma corretta di `execute()`. Il campo `TestResult.source` con tipo `Literal["native", "external"]` propaga la distinzione fino al report, dove i risultati nativi e quelli di tool specializzati vengono resi con sezioni visivamente distinte all'interno dello stesso dominio.

Il contratto imposto da `BaseTest` a ogni test concreto si articola in due livelli. Il primo è la dichiarazione obbligatoria di `ClassVar` che descrivono staticamente il test: `test_id` (identificatore univoco che corrisponde all'ID nella metodologia), `domain` (dominio di assurance 0-7), `priority` (P0-P3), `strategy` (`BLACK_BOX`, `GREY_BOX`, `WHITE_BOX`), `depends_on` (lista di `test_id` predecessori per il DAG), `cwe_id` e `tags` (riferimenti normativi). Il registry usa `has_required_metadata()` per scartare le classi con `ClassVar` incompleti prima ancora che vengano istanziate. Il secondo livello è l'implementazione di `execute(target, ctx, store)`, il metodo che non deve mai propagare eccezioni, come già discusso nella Sezione 4.1.1.

4.3.2 Auth Abstraction Layer e Idempotenza dei Token (D2.P5)

I test che operano in modalità `GREY_BOX` richiedono token validi per uno o più ruoli prima di poter raggiungere la logica applicativa che intendono sondare. La gestione di questi token è centralizzata nel modulo `src/tests/helpers/auth.py`, che espone la funzione `acquire_tokens()`.

Il meccanismo è idempotente per costruzione. Quando un test chiama `acquire_tokens(target, ctx)`, la funzione controlla prima se il `TestContext` contiene già token validi per i ruoli richiesti tramite `ctx.has_token(role)`. Solo se il token è assente esegue il login HTTP e scrive il risultato nel canale token del `TestContext` tramite `ctx.set_token(role, token)`. Con 18 test attivi in Milestone 1, di cui una parte significativa in modalità `GREY_BOX`, questa logica garantisce che il numero di login HTTP verso il target sia esattamente uguale al numero di ruoli distinti configurati, non al numero di test che li richiedono.

Il dispatcher in `auth.py` seleziona il meccanismo di autenticazione corretto in base al campo `auth_method` del `config.yaml`: nella configurazione attuale, il metodo supportato è l'autenticazione tramite API key di Forgejo. Aggiungere il supporto per un nuovo metodo, ad esempio Open Authorization 2.0 (OAuth 2.0) con client credentials, richie-

de una nuova implementazione del flusso di login e un ramo aggiuntivo nel dispatcher: nessun test esistente deve essere modificato, perché nessun test conosce il meccanismo di autenticazione sottostante, solo il ruolo che richiede.

4.3.3 Tracciabilità Normativa e Distinzione Finding/InfoNote (D5.P1, D5.P2)

Ogni test porta con sé, come `ClassVar`, i riferimenti normativi della garanzia che verifica: `cwe_id` identifica la classe di vulnerabilità nel Common Weakness Enumeration, e `tags` raccoglie le categorie Open Web Application Security Project (OWASP), i riferimenti National Institute of Standards and Technology (NIST) e gli Request for Comments (RFC) pertinenti. Questi metadati non sono annotazioni decorative: vengono propagati fino al modello `Finding` tramite il campo `references: list[str]`, e il report HTML li riporta a fianco di ogni risultato. Un analista che legge un finding può risalire direttamente alla sezione della metodologia, alla categoria OWASP API Security, e al Common Weakness Enumeration (CWE) senza consultare documentazione esterna.

La tracciabilità presuppone una distinzione semantica precisa tra due tipi di annotazione che un test può produrre. Un `Finding` è evidenza di una violazione di sicurezza: compare solo in risultati `FAIL`, viene contato nel totale complessivo dei finding, e il suo contenuto deve essere sufficiente a giustificare la valutazione negativa. Un `InfoNote` è un'annotazione informativa su un risultato `PASS`: non cambia lo status, non conta come violazione, e serve a documentare contesto rilevante per l'analista senza alterare il verdetto. La distinzione non è estetica, è architetturale: i contatori di finding nel report aggregano esclusivamente oggetti `Finding`, rendendo il totale affidabile e non influenzabile da annotazioni informative.

Un caso d'uso concreto illustra il valore della distinzione. Il test 4.3 verifica la presenza di un meccanismo di circuit breaking sull'infrastruttura del gateway. Kong OSS, nella configurazione di Milestone 1, non implementa circuit breaking attivo ma espone health-check passivi che svolgono una funzione parzialmente equivalente. Il test registra `PASS` perché il requisito minimo è soddisfatto, e allega una `InfoNote` che documenta la differenza tra il compensating control osservato e il circuit breaking completo. L'analista ottiene un quadro preciso senza che il sistema produca né un falso `FAIL` né un `PASS` silenzioso che nasconde la sfumatura.

L'invariante che un `PASS` non può coesistere con `Finding` è enforced da D4.P9, discusso nella Sezione 4.2.3: il `model_validator` di `TestResult` garantisce la coerenza al momento della costruzione.

4.3.4 Safe HTTP Probing Policy e Oracle a Tre Esiti (D4.P7)

Ogni verifica di sicurezza che richiede l'invio di una richiesta HTTP verso un endpoint del target deve interpretare la risposta ricevuta in modo non ambiguo. Il problema non è banale: uno status **404 Not Found** su un endpoint parametrico come `/users/{id}/repos` può significare che il gateway ha rifiutato la richiesta per mancanza di autenticazione, che il path non esiste per il valore di `{id}` usato nel probe, o che l'endpoint è genuinamente non protetto ma il path era errato. Interpretare un **404** come **FAIL** in tutti i casi genera falsi positivi inaccettabili in un assessment che si propone come affidabile.

La policy adottata definisce un oracle a tre esiti applicato sistematicamente a ogni probe HTTP nei test nativi:

1. **ENFORCED**: la risposta è **401** o **403**. Il gateway ha applicato un controllo di accesso. Il controllo funziona.
2. **BYPASSED**: la risposta è **2xx**. Il gateway non ha applicato il controllo atteso. Il controllo è assente o mal configurato.
3. **INCONCLUSIVE**: qualsiasi altro status. Il risultato non è interpretabile con certezza sufficiente per produrre un **Finding**.

Solo un esito **BYPASSED** produce un **Finding** e contribuisce a un risultato **FAIL**. Un esito **INCONCLUSIVE** viene registrato nell'evidenza con il suo status effettivo, ma non altera il verdetto del test.

A questa logica di interpretazione si affianca la segregazione degli endpoint in due tier. Il Tier A comprende i path fissi della specifica, cioè endpoint senza parametri di path o con parametri che il sistema sa risolvere tramite seed configurato: la risposta è interpretabile perché il path è valido. Il Tier B comprende i path parametrici senza seed configurato, dove la risposta dipende dal valore del placeholder usato nel probe: questi endpoint producono tipicamente **INCONCLUSIVE** e vengono annotati come tali nell'evidenza, segnalando all'operatore che configurare il path seed risolverebbe l'ambiguità.

Un caso limite merita attenzione esplicita. I probe con metodo **DELETE** su endpoint parametrici presentano un rischio operativo: se il gateway è mal configurato e non applica autenticazione, una richiesta **DELETE** con un path valido potrebbe eliminare risorse reali sul target. La policy prevede un fallback sicuro: quando il path seed non è configurato per un endpoint **DELETE** parametrico, il probe viene eseguito con il valore letterale `"apiguard-probe"` come sostituto del parametro. Un path con quel valore non corrisponde a nessuna risorsa reale su alcun sistema, garantendo che il probe non causi effetti collaterali indesiderati anche in presenza di un gateway non autenticato.

4.4 Gerarchia dei Connettori e Integrazione Tool Esterni

I test della gerarchia `ExternalToolTest` delegano la raccolta di dati a tool specializzati: scanner di vulnerabilità, analizzatori TLS, fuzzer. Il layer dei connector è il punto in cui questa delega si traduce in codice: isola la logica di invocazione di ogni tool, normalizza il suo output in un modello comune, e si interpone tra il test e il dettaglio implementativo dell'integrazione. Senza questo layer, ogni test dovrebbe contenere la logica di subprocess management o di importazione della libreria, accoppiando la valutazione della garanzia di sicurezza alla meccanica dell'esecuzione del tool.

4.4.1 Gerarchia a Tre Livelli e Separazione Dati/Valutazione (D1.P5, D2.P3)

I connector seguono una gerarchia a tre livelli che segue lo stesso principio della dual test hierarchy: ogni classe eredita solo i meccanismi pertinenti al proprio pattern di integrazione.

`BaseConnector` in `src/connectors/base.py` è il primo livello: un Abstract Base Class (ABC) puro con un solo `ClassVar` (`TOOL_NAME: str`) e tre metodi astratti che costituiscono il contratto universale. `is_available()` verifica se il tool è utilizzabile nell'ambiente corrente; `get_version()` restituisce la versione del tool per fini di audit; `run(target_url, timeout_seconds)` esegue il tool e restituisce un `ConnectorResult`. Il parametro `timeout_seconds` è intenzionalmente privo di valore di default: ogni chiamante deve fornirlo esplicitamente, letto da `config.yaml`, rendendo impossibile dimenticarlo.

Il secondo livello si biforca in due direzioni distinte a seconda della natura del tool. `BaseSubprocessConnector` serve i tool invocati come processo di sistema: fornisce implementazioni concrete di `is_available()` e `get_version()` basate su subprocess, e il metodo protetto `_run_subprocess()` che gestisce l'esecuzione, il timeout e il parsing dell'output. La ricerca del binario avviene tramite una cascata a tre canali, documentata come D7.P1: binario locale nella directory `tools/` del repository, poi `shutil.which()` per il path di sistema, infine una variabile d'ambiente per i deployment in cui il tool è esposto come microservizio HTTP. `BaseLibraryConnector` serve invece i tool Python-native: usa `importlib.util.find_spec()` per verificare la disponibilità della libreria senza invocare alcun subprocess, e legge la versione tramite `importlib.metadata`. Le implementazioni concrete, `NucleiConnector` e `TestsslConnector` per il primo ramo, `SslyzeConnector` per il secondo, si limitano a dichiarare le `ClassVar` specifiche del tool e implementare `run()`.

La separazione tra raccolta dati e valutazione è il principio che governa il contratto di

`run()`. Il connector non decide se qualcosa è un **FAIL**: restituisce un `ConnectorResult`, modello Pydantic frozen che trasporta il campo `raw_output: dict[str, Any]`, `exit_code`, `execution_time_ms`, e `timed_out`. È l'`ExternalToolTest` che implementa `_evaluate()` e valuta il `ConnectorResult` contro il proprio oracle. Lo stesso `NucleiConnector` è quindi riutilizzabile da test con logiche di valutazione diverse: un test che usa nuclei per il shadow API discovery e un ipotetico secondo test che usa nuclei per injection scanning condividono la stessa istanza del connector senza che il connector conosca le differenze tra le due valutazioni.

4.4.2 Lifecycle dei Connector e Dependency Injection (D2.P4)

La Phase R4 dell'`ExternalTestRegistry`, introdotta nella Sezione 4.2.1, produce un effetto preciso sul lifecycle dei connector: `is_available()` viene chiamato esattamente una volta per tool, indipendentemente da quanti test dipendono da quel tool.

Il meccanismo si basa su due attributi di istanza di ogni `ExternalToolTest`. Il campo `_injected_connector` viene valorizzato dal registry con l'istanza del connector quando il tool è disponibile: i test del gruppo condividono la stessa istanza, costruita una sola volta. Il campo `_skip_reason_from_registry` viene valorizzato quando il tool è assente: ogni test del gruppo riceve la motivazione di skip già scritta, e il metodo `_run()` la controlla come prima operazione, restituendo **SKIP** immediatamente senza costruire il connector né interrogare il filesystem.

La conseguenza operativa misurabile è nel log di esecuzione. Con cinque test che dipendono da nuclei e il binario assente, il log produce un solo **WARNING** del tipo *"nuclei not found — 5 tests will SKIP"*, non cinque entry identiche. In un assessment con molti tool e molti test, questa differenza rende il log leggibile invece che ridondante.

4.4.3 Classificazione A/B, Path Seed e Catalogo dei Payload (D2.P6, D2.P7, D2.P8)

I connector implementati in Milestone 1 sono classificati in due categorie che guidano le priorità di sviluppo e l'interpretazione dell'output dell'assessment. I connector di Categoria A sono prerequisiti per la completezza dei test **HYBRID**: la loro assenza produce **SKIP** su garanzie che altrimenti sarebbero parzialmente coperte. I connector di Categoria B espandono la copertura su garanzie già verificate da test nativi, senza essere necessari per l'assessment di base. Nella Milestone 1, `NucleiConnector` e `TestsslConnector/SslyzeConnector` sono entrambi Cat A: il primo perché copre shadow API discovery tramite template specializzati non replicabili con soli probe HTTP; i secondi perché l'analisi TLS richiede negoziazione di protocollo che supera le capacità di un client HTTP standard.

Il path seed system risolve un problema strutturale dell'agnosticismo applicativo. La specifica OpenAPI descrive endpoint con parametri di path come `/users/{owner}/repos/{repo}`: senza sapere quali valori concreti esistono sul target, il sistema non può costruire un path valido per il probe. Un path non valido produce tipicamente 404, che l'oracle classifica **INCONCLUSIVE** per le ragioni descritte nella Sezione 4.3.4. Il file di seed, generato tramite il comando CLI `generate-seed` o configurato manualmente in `config.yaml`, fornisce valori concreti per i parametri di path dei target noti. Con il seed configurato per Forgejo, i path con `{owner}` e `{repo}` producono risposte interpretabili: **ENFORCED** se il gateway applica il controllo atteso, **BYPASSED** se non lo applica. La Sezione ?? riporta i dati quantitativi di questo effetto nell'assessment di Milestone 1.

I payload di attacco usati dai test nativi, come le sequenze di URL per il test SSRF e gli header malformati per i test di input validation, risiedono in moduli puri in `src/tests/data/`, separati dalla logica di valutazione. Aggiungere un nuovo vettore di attacco, ad esempio una nuova sequenza di redirect per bypass di validazione, richiede aggiungere una riga al catalogo: la logica del test che lo usa non deve essere modificata. In contesti enterprise dove i payload di attacco sono soggetti a un processo di approvazione separato dal codice applicativo, questa separazione non è un'opzione di stile ma un prerequisito operativo.

4.4.4 Isolamento della Dipendenza AGPL (D7.P2)

`sslyze`, la libreria Python per l'analisi TLS, è distribuita sotto licenza GNU Affero General Public License (AGPL) v3. Includere `sslyze` come dipendenza obbligatorio del core imporrebbe gli obblighi copyleft dell'AGPL a qualsiasi distribuzione di API-Guard Assurance, rendendo incompatibile la licenza Massachusetts Institute of Technology (MIT) del resto del codebase. La soluzione è dichiarare `sslyze` esclusivamente in `[project.optional-dependencies.sslyze]` del `pyproject.toml`: una dipendenza opzionale che richiede installazione esplicita.

Un'installazione standard tramite `pip install apiguard-assurance` non installa `sslyze` e non impone nessun obbligo copyleft. L'operatore che vuole abilitare il test `ext.1.5.sslyze` installa esplicitamente l'extra, accettando consapevolmente la licenza AGPL. A runtime, `SslyzeConnector` usa `importlib.util.find_spec("sslyze")` in `is_available()`: se la libreria non è installata, il metodo restituisce `False` e il test riceve **SKIP** con motivazione esplicita, senza errori di importazione. Il core del sistema non contiene nessun import condizionale di `sslyze`: l'isolamento è strutturale, non gestito con `try/except ImportError`.

Il pattern è replicabile per qualsiasi dipendenza con vincoli di licenza o peso computazionale elevato: l'architettura `BaseLibraryConnector` con discovery tramite `importlib` è

progettata esattamente per questo uso, e l'aggiunta di un nuovo extra opzionale richiede solo una voce in `pyproject.toml` e una nuova sottoclasse di `BaseLibraryConnector`.

4.5 Implementazione del Gateway Adapter e Teardown LIFO

L'architettura del `BaseGatewayAdapter` è stata descritta nella Sezione 3.2.4 a livello di interfaccia e di motivazione. Questa sezione ne completa il quadro con l'implementazione concreta, descrive i tre livelli di degradazione controllata che il sistema applica quando parti dell'infrastruttura non sono disponibili, e illustra il meccanismo di cleanup delle risorse create durante l'assessment.

4.5.1 KongGatewayAdapter: Accesso Read-Only all'Admin API

`KongGatewayAdapter` in `src/core/gateway/kong.py` realizza l'interfaccia `BaseGatewayAdapter` interrogando la Admin API di Kong DB-less tramite richieste HTTP GET. Una scelta implementativa merita esplicitazione: il componente usa `httpx` direttamente, senza passare per `SecurityClient`. La motivazione è architetturale. Le chiamate alla Admin API sono audit di configurazione dell'infrastruttura, non traffico di test verso il target applicativo; non devono apparire nell'`EvidenceStore`, non sono soggette alla policy di retry del `SecurityClient`, e appartengono a un confine di fiducia distinto da quello del API target. Usare un client `httpx` leggero e dedicato mantiene i due confini esplicitamente separati nel codice.

I metodi pubblici dell'adapter, `get_routes()`, `get_plugins()`, `get_services()`, `get_upstreams()`, e `get_plugin_by_name()`, delegano tutti a un metodo interno `_fetch_paginated()` che gestisce la paginazione della Admin API di Kong. Le risposte seguono il formato `{"data": [...], "next": "/path?offset=..."}`: il metodo segue il cursore `next` finché è non-nullo, aggregando i risultati in un'unica lista. In modalità DB-less tutti gli oggetti vengono restituiti in una singola pagina e `next` è sempre `null`, ma il meccanismo è corretto anche in caso di Kong con database in ambienti che lo prevedono.

Il metodo `check_connectivity()` viene invocato dall'engine durante la Phase 3 per verificare la raggiungibilità della Admin API prima di popolare `TargetContext.gateway`. Se la verifica fallisce, il campo rimane `None` e tutti i test `WHITE_BOX` transitano a `SKIP` tramite l'helper `_requires_admin_api()` di `BaseTest`. Qualsiasi errore sollevato dalle chiamate HTTP durante l'esecuzione dei test viene incapsulato in `GatewayAdapterError`, che il test cattura e converte in `TestResult(ERROR)` senza propagare l'eccezione all'engine.

4.5.2 Degradazione Controllata a Tre Livelli (D4.P5)

Un assessment di sicurezza eseguito in ambienti eterogenei incontra sistematicamente situazioni in cui parti dell'infrastruttura non sono disponibili: tool esterni non installati, Admin API non esposta, integrazione con tool esterni disabilitata per policy. La risposta corretta non è un errore a cascata che invalida l'intero run, ma una degradazione controllata che produce risultati parziali corretti con motivazioni esplicite.

Il sistema implementa tre livelli distinti di degradazione, ciascuno con un meccanismo specifico.

Il primo livello è il master switch `external_tools.enabled` in `config.yaml`. Quando impostato a `false`, l'`ExternalTestRegistry` restituisce una lista vuota senza scandagliare il filesystem né importare moduli: zero overhead, zero SKIP nel log, comportamento completamente silenzioso. Questo livello serve ambienti in cui la presenza di tool esterni è impossibile per policy di sicurezza o di deployment.

Il secondo livello gestisce i tool esterni assenti a livello di singolo tool. Come illustrato nella Sezione 4.4.2, la Phase R4 dell'`ExternalTestRegistry` imposta `_skip_reason_from_registry` su ogni test del gruppo il cui tool non è disponibile: ogni test riceve SKIP con motivazione esplicita, un solo WARNING nel log per tool.

Il terzo livello gestisce l'assenza della Admin API del gateway. Se `gateway_adapter` non è configurato in `config.yaml` oppure se `check_connectivity()` fallisce, `TargetContext.gateway` rimane `None`. Ogni test `WHITE_BOX` chiama come prima operazione `self._requires_admin_api(target)`: se il campo è `None`, restituisce immediatamente SKIP con il testo *"Admin API not configured"* senza eseguire nessuna logica di verifica. Il risultato dell'assessment in questo scenario copre i soli domini black-box e grey-box, con un numero di SKIP equivalente al numero di test white-box presenti nel set attivo.

La distinzione tra i tre livelli di SKIP è visibile nel report: il campo `skip_reason` di ogni `TestResult` porta la motivazione specifica, rendendo immediata la diagnosi di quale componente mancante ha ridotto la copertura dell'assessment.

4.5.3 Teardown Best-Effort in Ordine LIFO (D4.P6)

I test che raggiungono la logica applicativa del target spesso creano risorse necessarie alla verifica: token di autenticazione temporanei, utenti di test, risorse effimere. Lasciare queste risorse sul target al termine dell'assessment viola la proprietà di non-interferenza: un assessment successivo potrebbe incontrare uno stato alterato, e il target produttivo potrebbe accumulare risorse orfane nel tempo.

Il meccanismo di teardown è registrazione esplicita al momento della creazione. Ogni volta che un test effettua un'operazione che crea una risorsa persistente, chiama immediatamente `ctx.register_resource_for_teardown(method, path, headers)` passando l'endpoint di cancellazione. La registrazione avviene nella stessa transazione logica della creazione: se il test va in eccezione dopo la creazione ma prima di completare la registrazione, la risorsa potrebbe non essere rimossa. Questa condizione è documentata come limite noto e non è considerabile dai meccanismi di eccezione di D4.P3.

In Phase 6, l'engine chiama `ctx.drain_resources()` che restituisce le risorse registrate in ordine LIFO. L'ordine inverso rispetto alla creazione garantisce che risorse con dipendenze implicite vengano rimosse correttamente: una risorsa creata dopo un'altra e che dipende da essa viene eliminata per prima. Per ogni risorsa restituita, l'engine tenta la chiamata HTTP di cancellazione tramite `SecurityClient`. Un fallimento solleva `TeardownError`, che viene catturato e registrato come `WARNING` strutturato: il fallimento del teardown è un avvertimento operativo, non un'invalidazione scientifica dei risultati dell'assessment che lo ha preceduto. Come dimostrato nell'audit di Milestone 1 e discusso nella Sezione ??, zero risorse sono state rilevate come non rimosse su due run indipendenti.

4.5.4 Astrazione Trasparente dell'URL di Deployment (D7.P3)

Un assessment eseguito in sviluppo locale raggiunge il target all'URL `http://localhost:8080`; lo stesso assessment eseguito in un ambiente Docker Compose deve usare il nome del servizio Compose, ad esempio `http://api-gateway:8080`, per essere raggiungibile da un container che condivide la stessa rete. In assenza di un meccanismo di astrazione, ogni connector che costruisce URL di destinazione conterrebbe logica condizionale di deployment.

Il campo `effective_base_url` di `TargetContext` risolve questo problema una volta sola in Phase 3. L'engine legge `target.base_url` dal `config.yaml`, verifica se è presente la variabile d'ambiente `APIGUARD_TARGET_EFFECTIVE_URL` (usata per override nei deployment Docker), e scrive il risultato nel campo frozen di `TargetContext`. Da quel momento, tutti i connector usano `target.effective_base_url` tramite il metodo `target.effective_endpoint_base_url()`: zero logica condizionale distribuita nei connector, zero differenze tra i deployment visibili al di fuori di Phase 3.

4.6 Evidence Store e Sanitizzazione delle Credenziali

Ogni verifica di sicurezza produce due tipi di output: un verdetto, espresso come `TestResult`, e l'evidenza che lo sostiene, composta dalle transazioni HTTP effettuate e dai dati grez-

zi prodotti dai tool esterni. Il verdetto senza evidenza è un’asserzione; l’evidenza senza sanitizzazione è un rischio. Questa sezione descrive come `EvidenceStore` gestisce entrambi i problemi, e come i due deliverable finali dell’assessment, il file `evidence.json` e il report `assessment_report.html`, vengono prodotti e strutturati.

4.6.1 Ciclo di Vita dello Streaming Evidence Store (D4.P1)

La motivazione architetturale dell’`EvidenceStore` è stata introdotta nella Sezione 3.2.5: il design precedente basato su `deque(maxlen=100)` produceva audit trail rotti in silenzio. Qui si descrive l’implementazione concreta che ha sostituito quel design.

Il ciclo di vita dell’`EvidenceStore` si articola in quattro operazioni sincronizzate con le fasi dell’engine. In Phase 3, l’engine istanzia `EvidenceStore` e crea la directory temporanea `outputs/evidence_tmp/`. Prima di eseguire ogni test in Phase 5, l’engine chiama `store.begin_test(test_id)`: il metodo apre il file `outputs/evidence_tmp/{test_id}.jsonl` in modalità append e lo mantiene aperto per l’intera durata del test. Durante l’esecuzione, ogni chiamata a `store.log_transaction()` o `store.pin_artifact()` scrive immediatamente un oggetto JSON su una singola riga del file con `flush()` esplicito: zero buffering in memoria, zero rischio di perdita in caso di crash. Al termine del test, `store.end_test()` chiude il file. In Phase 7, `store.merge_and_finalize()` legge tutti i file `.jsonl` dalla directory temporanea, li ordina per `test_id` lessicografico per garantire un output deterministico, li concatena in un unico array JSON e li scrive nel file definitivo `outputs/evidence.json`. La directory temporanea viene poi rimossa.

Il vantaggio rispetto al design precedente è eliminare la classe di bug per costruzione. Con `deque(maxlen=100)`, il primo record di un test con più di 100 transazioni veniva silenziosamente rimosso dalla coda, generando `Finding.evidence_ref` che puntavano a record inesistenti nell’audit trail. Con il design corrente, ogni record scritto su disco è permanente. Il test 1.1 su Forgejo, che interroga oltre 100 endpoint documentati nella specifica OpenAPI, produce un file `1.1.jsonl` con un record per endpoint senza nessuna perdita: la dimensione dell’evidenza è limitata unicamente dalla costante nominata `RESPONSE_BODY_MAX_CHARS = 10 000` che tronca i body di risposta singoli, non il numero di record.

La proprietà di recuperabilità è un effetto collaterale dell’architettura. Se il processo va in crash tra Phase 5 e Phase 7, i file `.jsonl` rimangono su disco e sono leggibili manualmente con qualsiasi strumento JSONL-compatibile: l’evidenza parziale è recuperabile anche in assenza di un run completo.

4.6.2 Sanitizzazione Centralizzata delle Credenziali (D4.P2)

Un audit trail che contiene token di autenticazione, API key o password è esso stesso una vulnerabilità: il file `evidence.json` viene condiviso con il team di sicurezza, archiviato nei sistemi di ticketing, e potenzialmente incluso in repository. La sanitizzazione delle credenziali non può essere delegata ai singoli connector con l'aspettativa che ciascuno ricordi di redactare i campi sensibili: in un sistema estensibile da terze parti, quella convenzione non è verificabile staticamente e non è garantibile a lungo termine.

La risposta adottata è centralizzare la responsabilità in un unico punto. Il metodo `EvidenceStore._sanitize_artifact()` viene invocato automaticamente da `pin_artifact()` su qualsiasi dato prodotto da connector esterni prima che venga scritto su disco. Il connector che produce il dato non è tenuto a fare nulla: la garanzia è strutturale, non convenzionale.

La sanitizzazione opera con tre meccanismi sovrapposti che si completano a vicenda. Il primo è il key-pattern matching: una regex compilata a word-boundary controlla ogni chiave del dizionario ricorsivamente rispetto a 11 pattern (`token`, `password`, `api_key`, `apikey`, `authorization`, `bearer`, `secret`, `credential`, `auth`, `private_key`, `access_token`). Qualsiasi chiave che corrisponde a uno di questi pattern vede il proprio valore sostituito con `[REDACTED]`, indipendentemente dal tipo del valore.

Il secondo meccanismo è la JWT value detection: la regex `_SANITIZE_JWT_PATTERN` riconosce la struttura a tre segmenti base64url separati da punto che caratterizza un JWT e redacta qualsiasi stringa con quella forma, indipendentemente dalla chiave che la contiene. Questo cattura i token che appaiono come valori di chiavi con nomi arbitrari, ad esempio in un campo `result` del raw output di un tool.

Il terzo meccanismo è l'header prefix matching: qualsiasi stringa che inizia con `"Bearer "`, `"Basic "`, `"Token "`, o `"token "` viene redactata, catturando i token che appaiono nel valore di header HTTP inclusi nei record di evidenza.

La tripla copertura garantisce che un token JWT leakato come valore di una chiave `"result"` arbitraria in un tool non noto al sistema venga comunque redactato, perché il secondo meccanismo opera sul valore indipendentemente dalla chiave. Un connector futuro contribuito da terze parti che dimentica di redactare i propri campi sensibili non crea una violazione nell'audit trail: la garanzia è nel punto di scrittura su disco, non nel punto di produzione del dato.

4.6.3 Dual Audit Trail e Report Domain-Centric (D5.P5)

I due deliverable finali dell'assessment assolvono funzioni complementari e si rivolgono a interlocutori distinti. `evidence.json` è il forensic trail tecnico: contiene ogni transazione HTTP, ogni artifact di tool esterno, ogni record di finding con la sua evidenza di supporto, in formato macchina nativamente ingestibile da log aggregator come Elasticsearch, Splunk o Loki. `assessment_report.html` è il deliverable operativo: generato da `report/render.py` tramite Jinja2, è leggibile da browser senza dipendenze esterne e riporta i risultati in linguaggio comprensibile a un team di sicurezza non specializzato sullo specifico tool.

La struttura del report è determinata dal modo in cui l'`EvidenceStore` aggrega i dati durante la Phase 7. Il `ReportBuilder` in `report/builder.py` raggruppa i `TestResult` per coppia (`domain`, `source`): per ogni dominio di assurance compaiono prima i risultati dei test nativi, con la loro evidenza HTTP diretta, poi i risultati dei test con tool specializzati, con il raw output del tool. Questa partizione non richiede sezioni fisicamente separate nel template: emerge naturalmente dall'aggregazione dei dati e dal campo `TestResult.source` che distingue "native" da "external".

Il report include anche la transaction log di ogni test: la lista ordinata di tutte le transazioni HTTP effettuate durante l'esecuzione, incluse le richieste e le risposte con i body troncati alla costante `RESPONSE_BODY_MAX_CHARS`. Questa sezione, renderizzata con lazy loading nel template HyperText Markup Language (HTML) per gestire report di grandi dimensioni, permette a un analista di ripercorrere l'intera sequenza di probe che ha prodotto un finding, riproducendo il ragionamento del tool senza dover rieseguire l'assessment.

Capitolo 5

Validazione sperimentale e risultati

5.1 Setup sperimentale

5.2 Risultati

Capitolo 6

Discussione e Sviluppi Futuri

6.1

Capitolo 7

Conclusioni

7.1 Conclusioni

7.2 Lavori futuri

Bibliografia

- [1] “OWASP Top Ten Web Application Security Risks | OWASP Foundation.” [Online]. Available: <https://owasp.org/www-project-top-ten/>